

# Introduction to Matplotlib and Line Plots

## What I'm Doing

This lab introduces **data visualization in Python** using Matplotlib and pandas. I'll work with Canadian immigration dataset from 1980–2013, setting up the environment, exploring the data, cleaning it up, and then creating **line plots** to uncover meaningful trends.

---

## 1. Setting Up

First, I install the three essential libraries — `pandas` for data wrangling, `numpy` for numerical work, and `matplotlib` for plotting.

```
%pip install pandas
%pip install numpy
%pip install matplotlib
```

Once installed, I import them. I'll also configure Matplotlib's inline backend (so plots display directly in the notebook) and apply the `ggplot` style for a cleaner look.

```
import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt

%matplotlib inline

# Optional: check version
print('Matplotlib version:', mpl.__version__)

# Apply a ggplot-like style
print(plt.style.available)
mpl.style.use(['ggplot'])
```

```
Matplotlib version: 3.10.9
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'petroff10', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```

**Result:** The style list confirms `ggplot` is available and I've set it as the active theme.

Now I download the Canadian immigration dataset directly from its hosted URL using `pandas.read_csv()`.

```
df_can = pd.read_csv(
    'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/'
    'IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/'
```

```
'Data%20Files/Canada.csv'  
)
```

The data is now loaded and ready to explore.

---

## 2. Getting to Know the Dataset

Let's check what we're working with by looking at the first few rows.

```
df_can.head()
```

|   | Country        | Continent | Region          | DevName            | 1980 | 1981 | 1982 | 1983 | 1984 | 1985 | ... | 2005 | : |
|---|----------------|-----------|-----------------|--------------------|------|------|------|------|------|------|-----|------|---|
| 0 | Afghanistan    | Asia      | Southern Asia   | Developing regions | 16   | 39   | 39   | 47   | 71   | 340  | ... | 3436 |   |
| 1 | Albania        | Europe    | Southern Europe | Developed regions  | 1    | 0    | 0    | 0    | 0    | 0    | ... | 1223 |   |
| 2 | Algeria        | Africa    | Northern Africa | Developing regions | 80   | 67   | 71   | 69   | 63   | 44   | ... | 3626 |   |
| 3 | American Samoa | Oceania   | Polynesia       | Developing regions | 0    | 1    | 0    | 0    | 0    | 0    | ... | 0    |   |
| 4 | Andorra        | Europe    | Southern Europe | Developed regions  | 0    | 0    | 0    | 0    | 0    | 0    | ... | 0    |   |

5 rows × 39 columns

**Result:** The dataset contains **39 columns** Country , Continent , Region , DevName (development name), 1980 through 2013 (34 yearly columns), Total (sum of all years)

---

## 3. Cleaning the Data

A clean dataset makes plotting much smoother. Let's set the Country column as the index, which lets us reference countries by name directly when plotting.

```
df_can.set_index('Country', inplace=True)
```

```
# View the result  
df_can.head(3)
```

|                    | Continent | Region          | DevName            | 1980 | 1981 | 1982 | 1983 | 1984 | 1985 | 1986 | ... | 201 |
|--------------------|-----------|-----------------|--------------------|------|------|------|------|------|------|------|-----|-----|
| Country            |           |                 |                    |      |      |      |      |      |      |      |     |     |
| <b>Afghanistan</b> | Asia      | Southern Asia   | Developing regions | 16   | 39   | 39   | 47   | 71   | 340  | 496  | ... | 34  |
| <b>Albania</b>     | Europe    | Southern Europe | Developed regions  | 1    | 0    | 0    | 0    | 0    | 0    | 1    | ... | 12  |
| <b>Algeria</b>     | Africa    | Northern Africa | Developing regions | 80   | 67   | 71   | 69   | 63   | 44   | 69   | ... | 36  |

3 rows × 38 columns

**Result:** The index is now `Country`. I'll also remove the name label from the index for a cleaner display.

```
df_can.index.name = None
```

Now I'll create a helper variable that stores the years as strings — this makes it easy to reference the year columns throughout the notebook.

```
years = list(map(str, range(1980, 2014)))
years
```

**Result:**

```
['1980', '1981', '1982', '1983', '1984', '1985', '1986', '1987', '1988',
 '1989',
 '1990', '1991', '1992', '1993', '1994', '1995', '1996', '1997', '1998',
 '1999',
 '2000', '2001', '2002', '2003', '2004', '2005', '2006', '2007', '2008',
 '2009',
 '2010', '2011', '2012', '2013']
```

The data is now cleaned and organized. Every country is indexed by name, and I have a ready-to-use list of year columns for slicing and plotting.

## 4. Extracting and Plotting a Single Country (Haiti)

With the data in shape, I can start creating line plots. Let's begin with a **case study on Haiti**.

In 2010, Haiti was hit by a catastrophic magnitude 7.0 earthquake, devastating the country and affecting roughly three million people. Canada responded by stepping up refugee acceptance — and the immigration data should reflect that.

**Step 1: Extract the series for Haiti.**

```
haiti = df_can.loc['Haiti', years]
haiti.head()
```

```
1980    1666
1981    3692
1982    3498
1983    2860
1984    1418
Name: Haiti, dtype: int64
```

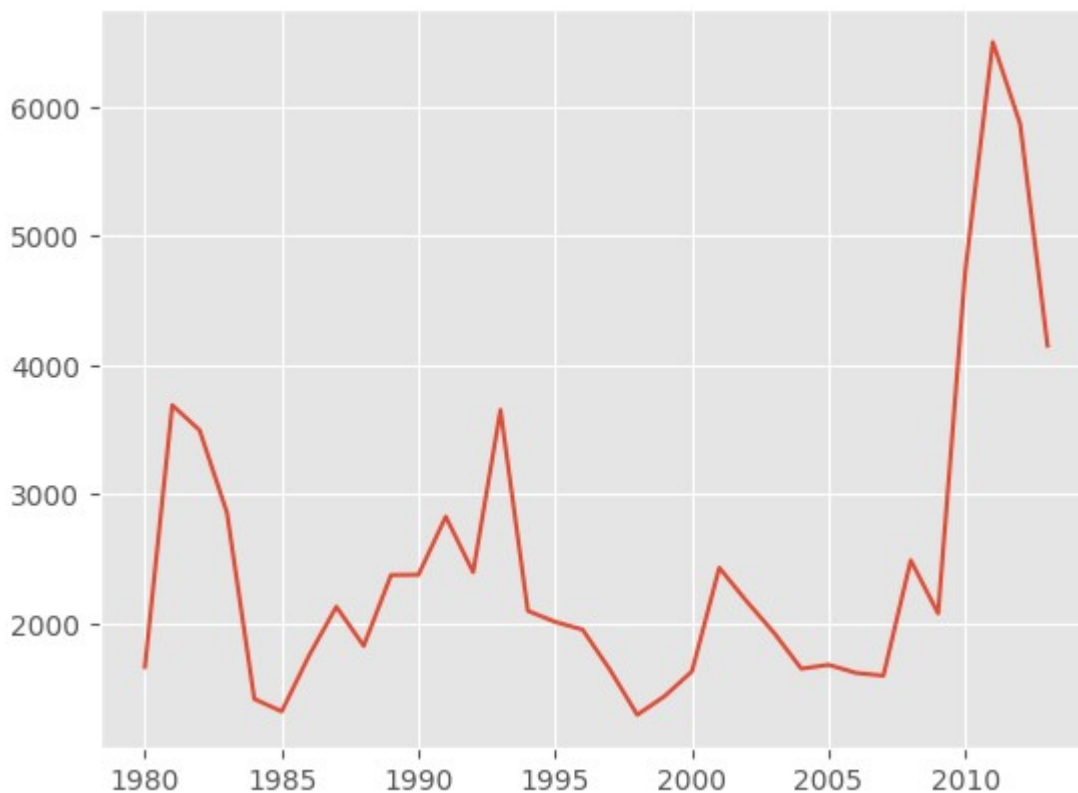
**Result:** Haiti is now a pandas **Series** with years as the index and immigrant counts as the values.

## Step 2: Plot the line.

```
haiti.plot(kind='line')

plt.title('Immigration from Haiti')
plt.ylabel('Number of immigrants')
plt.xlabel('Years')

plt.show(); #adding semicolon (;) to suppress "<Axes: >" text
```



**Result:** The spike after 2010 is unmistakable. Let's make it even clearer by annotating the earthquake event.

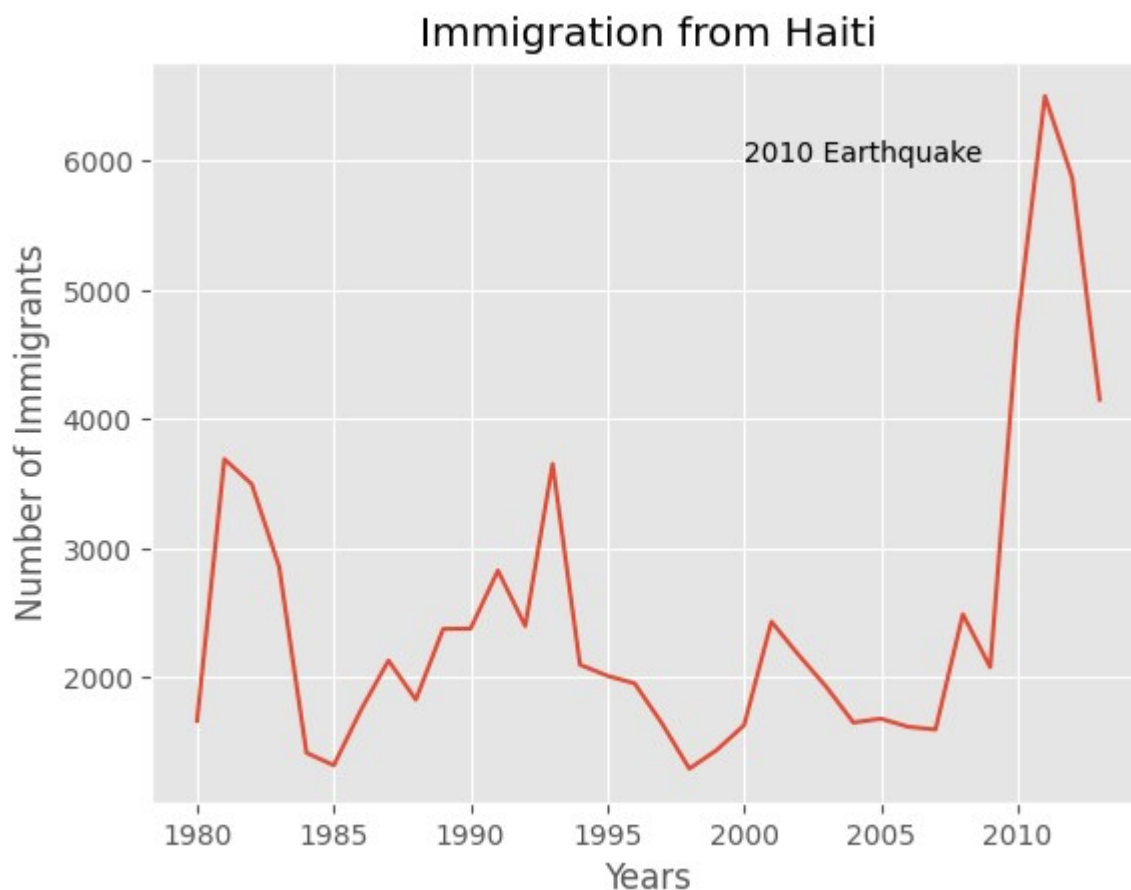
```
# Convert index from string to int for better text positioning
haiti.index = haiti.index.map(int)

haiti.plot(kind='line')

plt.title('Immigration from Haiti')
plt.ylabel('Number of Immigrants')
plt.xlabel('Years')

# Annotate the 2010 Earthquake
plt.text(2000, 6000, '2010 Earthquake') # (x-pos, y-pos, "Text")
```

```
plt.show()
```



**Result:** a text annotation "2010 Earthquake" placed at the year 2000, pointing toward the post-2010 surge.

## Exercise: Comparing Immigration from India and China

Let's compare two major immigrant-source countries — India and China — over the same period.

### Step 1: Extract the data.

```
df_CI = df_can.loc[['India', 'China'], years]
df_CI
```

|       | 1980 | 1981 | 1982 | 1983 | 1984 | 1985 | 1986 | 1987  | 1988  | 1989  | ... | 2004  | 2005  | 2006  |
|-------|------|------|------|------|------|------|------|-------|-------|-------|-----|-------|-------|-------|
| India | 8880 | 8670 | 8147 | 7338 | 5704 | 4211 | 7150 | 10189 | 11522 | 10343 | ... | 28235 | 36210 | 33848 |
| China | 5123 | 6682 | 3308 | 1863 | 1527 | 1816 | 1960 | 2643  | 2758  | 4323  | ... | 36619 | 42584 | 33518 |

2 rows × 34 columns

### Step 2: Transpose and plot.

Because `df_CI` has countries as rows and years as columns, I need to **transpose** it so that years become the x-axis and each country becomes a separate line.

```
df_CI = df_CI.transpose()
df_CI.head()
```

|      | India | China |
|------|-------|-------|
| 1980 | 8880  | 5123  |
| 1981 | 8670  | 6682  |
| 1982 | 8147  | 3308  |
| 1983 | 7338  | 1863  |
| 1984 | 5704  | 1527  |

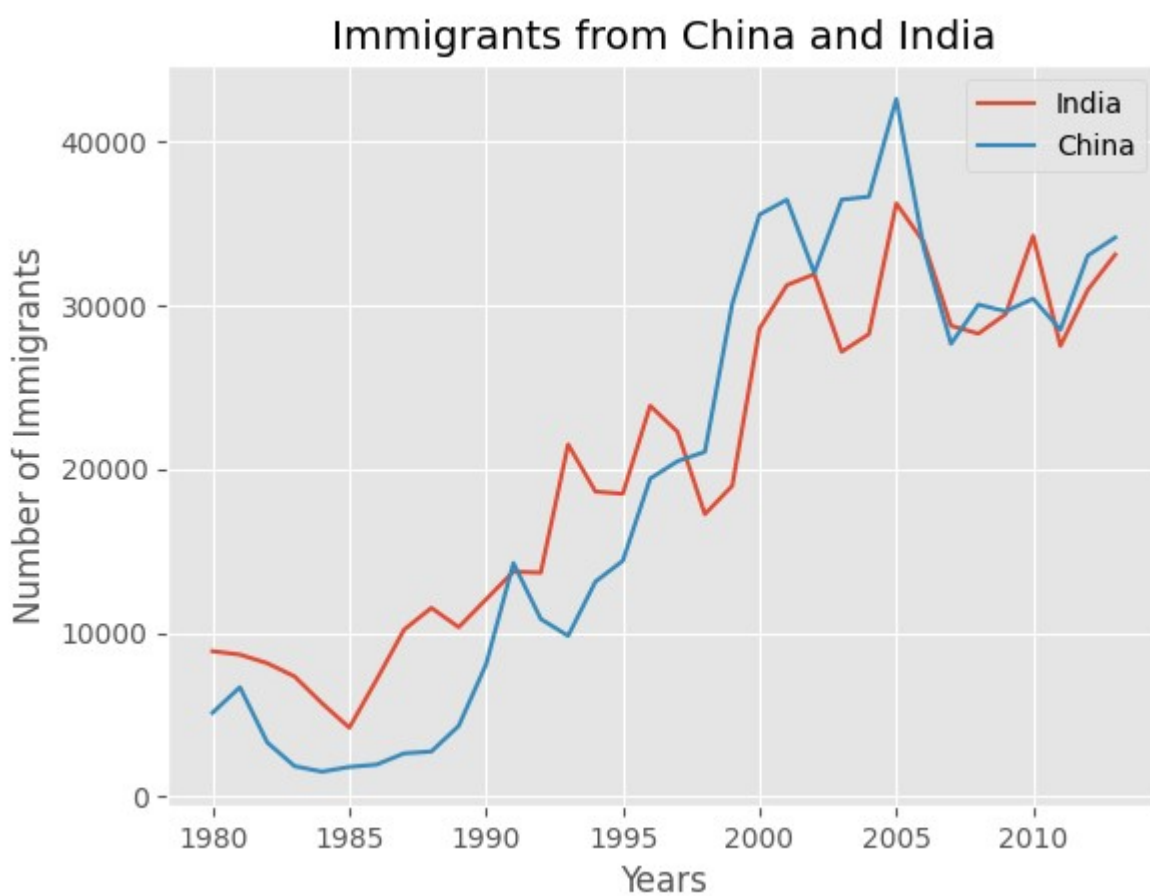
Now plot:

```
df_CI.index = df_CI.index.map(int) # convert to integers for clean x-axis

df_CI.plot(kind='line')

plt.title('Immigrants from China and India')
plt.ylabel('Number of Immigrants')
plt.xlabel('Years')

plt.show()
```



**Result:** China and India have very similar immigration trends through the years.

**Why didn't we need to transpose Haiti's data?**

Because `haiti` was a **Series** (one-dimensional), with years already in its index. A DataFrame with multiple columns (India and China) needs transposing so each column becomes a separate plotted line.

---

## Exercise: Top 5 Countries Contributing the Most Immigrants

Now let's zoom out and look at the top 5 countries overall.

### Step 1: Sort by Total and grab the top 5.

```
df_can.sort_values(by='Total', ascending=False, axis=0, inplace=True)

# Get the top 5 entries
df_top5 = df_can.head(5)

# Transpose the yearly data
df_top5 = df_top5[years].transpose()

print(df_top5.head(5))
```

|      | India | China | United Kingdom of Great Britain and Northern Ireland | \     |
|------|-------|-------|------------------------------------------------------|-------|
| 1980 | 8880  | 5123  |                                                      | 22045 |
| 1981 | 8670  | 6682  |                                                      | 24796 |
| 1982 | 8147  | 3308  |                                                      | 20620 |
| 1983 | 7338  | 1863  |                                                      | 10015 |
| 1984 | 5704  | 1527  |                                                      | 10170 |

|      | Philippines | Pakistan |
|------|-------------|----------|
| 1980 | 6051        | 978      |
| 1981 | 5921        | 972      |
| 1982 | 5249        | 1201     |
| 1983 | 4562        | 900      |
| 1984 | 3801        | 668      |

### Step 2: Plot with a larger figure size for readability.

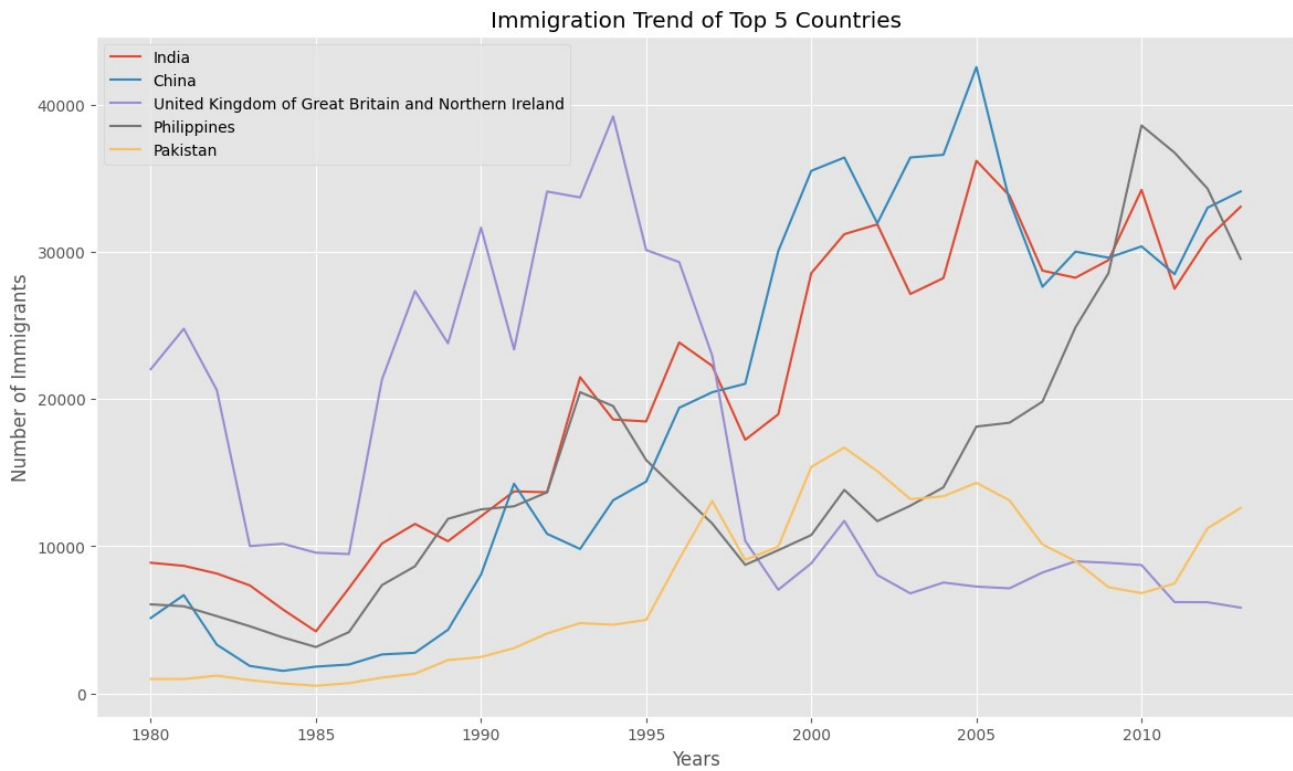
```
df_top5.index = df_top5.index.map(int)

df_top5.plot(kind='line', figsize=(14, 8))

plt.title('Immigration Trend of Top 5 Countries')
plt.ylabel('Number of Immigrants')
plt.xlabel('Years')

plt.show()
```





**Result:** This visualization immediately reveals the **shifting sources** of Canadian immigration over three decades: a decline from traditional European sources and a surge from Asia.

Line plot is a handy tool to display several dependent variables against one independent variable. However, it is recommended that no more than 5-10 lines on a single graph; any more than that and it becomes difficult to interpret.

That's the full walkthrough. The dataset is now clean, indexed by country, and explored with line plots highlighting trends like Haiti's spike, India and China's rise, and shifting immigration sources — ready for deeper visualization and analysis